PredLDA

Analyse discriminante linéaire prédictive en R

M2 SISE — Programmation R — 2026–2027

Thomas Fonferrier Ryan Ducret Aya Saadi Xilian Jing
Équipe G

13 novembre 2026

Résumé

Ce rapport présente le package R **PredLDA**. Le package propose une analyse discriminante linéaire prédictive, accessible via une classe R6 proche de l'API scikit-learn. Nous décrivons les formules utilisées, le découpage du code, le déroulement de l'apprentissage et de la prédiction, l'application Shiny, les tests, puis l'installation depuis GitHub. Les captures de l'interface figurent en annexe.

Table des matières

1 Introduction **2**

2 Méthode : LDA prédictive **2**
- 2.1 Principe . . . 2
- 2.2 Fonctions de classification . . . 2
- 2.3 Estimation de W . . . 2
- 2.4 Priors . . . 2
- 2.5 Cas à deux classes . . . 2

3 Architecture du package **3**
- 3.1 Interface R6 . . . 3

4 Pseudo-code **3**
- 4.1 Apprentissage . . . 3
- 4.2 Prédiction . . . 3

5 Choix techniques **4**

6 Application Shiny **4**

7 Tests **4**

8 Installation et documentation **4**

9 Limites **5**

10 Conclusion **5**

A Modules exportés **5**

B Jeux d'exemple **5**

1 Introduction

Le projet consiste à livrer un package R installable depuis GitHub. Le package doit proposer une LDA prédictive pour une cible à au moins deux classes. Il doit aussi exposer les fonctions de classification, fournir une documentation en anglais, un tutoriel reproductible et une application Shiny.

Le code est hébergé ici : `https://github.com/thomas-fonferrier/projet-lda-r`. Le package installable se trouve dans le dossier `PredLDA/`.

2 Méthode : LDA prédictive

2.1 Principe

On dispose de variables numériques X et d'une cible y à K classes. Pour chaque classe k, on estime une moyenne μ_k et une probabilité a priori π_k. On estime aussi une covariance intra-classe commune W. Sous cette hypothèse, la règle de décision se ramène à des scores linéaires [1].

2.2 Fonctions de classification

Pour chaque classe k :

$$a_k = W^{-1}\mu_k, \tag{1}$$

$$a_{0k} = \log \pi_k - \tfrac{1}{2}\mu_k^\top W^{-1}\mu_k, \tag{2}$$

$$d_k(x) = a_{0k} + a_k^\top x. \tag{3}$$

On affecte l'observation à la classe de score maximal. Les probabilités affichées par `predict_proba` viennent d'un softmax sur ces scores.

2.3 Estimation de W

Après centrage de chaque ligne sur la moyenne de sa classe, on calcule

$$W = \frac{X_c^\top X_c}{n - K}. \tag{4}$$

Si W n'est pas inversible, le package utilise une pseudo-inverse et signale un avertissement.

2.4 Priors

Par défaut, $\pi_k = n_k/n$. On peut aussi fournir des priors personnalisés. Ils n'agissent que sur l'intercept, via $\log \pi_k$.

2.5 Cas à deux classes

Le calcul reste le même qu'en multi-classes. Pour $K = 2$, `summary()` affiche aussi $D(x) = d_1(x) - d_2(x)$. C'est une lecture plus simple de la frontière, pas une autre méthode.

3 Architecture du package

Le code est séparé en deux couches. Le fichier `stats_utils.R` contient les calculs. Le fichier `lda_r6.R` contient la classe publique, les contrôles d'entrée et l'affichage.

Élément	Rôle
`R/stats_utils.R`	Calculs internes
`R/lda_r6.R`	Classe `LDA`
`man/`	Aide `?LDA`
`tests/`	Tests et comparaison à `MASS::lda`
`inst/extdata/`	Jeux d'exemple
`inst/shiny/`	Application Shiny
`vignettes/`	Tutoriel installable
`report/`	Sources de ce rapport

TABLE 1 – Órganisation du package PredLDA.

L'application Shiny n'implémente pas l'algorithme. Elle appelle `fit`, `predict` et `predict_proba`.

3.1 Interface R6

— `LDA$new(priors, tol)` : création du modèle ;

— `$fit(X, y)` : apprentissage ;

— `$predict(X)` et `$predict_proba(X)` : déploiement ;

— `$score(X, y)` : exactitude ;

— `$print()` et `$summary()` : lecture des résultats.

Après `fit`, le modèle expose notamment `coefficients`, `means`, `priors`, `within_cov`, `classes` et `feature_names`.

Dans `coefficients`, chaque colonne correspond à une classe. La première ligne est l'intercept. À la prédiction, les colonnes de X sont alignées par nom.

4 Pseudo-code

4.1 Apprentissage

```
FIT(X, y; priors_user, tol):
  valider X (numerique, fini, sans NA)
  valider y (>= 2 classes, n > K)
  M  <- moyennes par classe
  W  <- covariance poolee
  W_inv <- inverse(W) ou pseudo-inverse
  pi <- priors (defaut = proportions)
  pour chaque classe k:
    a_k  <- W_inv * mu_k
    a0_k <- log(pi_k) - 0.5 * mu_k' * W_inv * mu_k
  stocker coefficients, means, priors, W
```

4.2 Prédiction

```
DECISION_FUNCTION(X):
  aligner les colonnes de X sur feature_names
  retourner a0 + X %*% A
```

```
PREDICT(X):
  retourner la classe de score maximal

PREDICT_PROBA(X):
  retourner softmax des scores
```

Le softmax retire d'abord le maximum de chaque ligne. Cela limite les problèmes numériques quand les scores sont très éloignés.

5 Choix techniques

L'inversion de W passe d'abord par une décomposition de Cholesky. Si cela échoue, on bascule sur une décomposition en valeurs propres.

Le package dépend peu de librairies externes. Seul `R6` est requis pour l'usage courant. `MASS` sert surtout aux tests de référence. `shiny` et `readxl` restent optionnels.

6 Application Shiny

L'application se lance après installation du package :

```
shiny::runApp(system.file("shiny", package = "PredLDA"))
```

Elle permet de charger des données, de choisir la cible et les variables explicatives, d'ajouter des variables illustratives, d'entraîner le modèle, puis d'évaluer et de prédire. Deux modes d'évaluation sont proposés : sur l'apprentissage, ou avec un partage stratifié 80/20 (graine fixe 2026).

Les captures du parcours sur l'exemple iris sont regroupées en annexe C. Elles montrent le choix des données, l'ajustement, les coefficients, l'évaluation et la prédiction d'une nouvelle observation.

Un tutoriel vidéo présente aussi le fonctionnement de l'application : *Tutoriel - App R Shiny - Package PredLDA - Master SISE 2026/2027*, disponible à l'adresse `https://youtu.be/QUwlcWtNWKk`.

7 Tests

Deux fichiers principaux couvrent le cœur du package. `tests/test-LDA.R` vérifie l'API, les erreurs attendues et l'accord avec `MASS::lda` sur iris. `tests/test-reference.R` compare les coefficients à une formule indépendante, sur plusieurs jeux de données. Un test hors échantillon utilise aussi Pima.

Sur ces contrôles, les postérieurs restent alignés avec `MASS::lda` [2] à 10^{-8} près.

8 Installation et documentation

Installation depuis GitHub :

```
remotes::install_github(
  "thomas-fonferrier/projet-lda-r",
  subdir = "PredLDA",
  upgrade = "never"
)
```

Le paramètre `subdir` est nécessaire car le package n'est pas à la racine du dépôt. Une archive `PredLDA_0.1.0.tar.gz` sert de secours si l'installation en ligne n'est pas possible.

La documentation utilisateur comprend :

— l'aide `?LDA`, rédigée en anglais ;

— le README du package et la vignette `predlda-intro` ;

— les fichiers d'exemple placés dans `inst/extdata/`.

Un contrôle `R CMD check` a été exécuté avec succès. Une action GitHub relance ce contrôle à chaque mise à jour de la branche principale.

9 Limites

Le cœur actuel n'accepte que des variables explicatives numériques. Le traitement des variables catégorielles, la sélection automatique de variables et des indicateurs de pertinence restent des extensions possibles. Dans Shiny, le choix manuel des variables ne remplace pas une procédure automatique de sélection.

10 Conclusion

PredLDA fournit une LDA prédictive utilisable depuis R et depuis une interface Shiny. L'implémentation suit les formules classiques [1]. Elle a été confrontée à `MASS::lda`. Le dépôt permet une installation directe, avec documentation et exemples.

Références

[1] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning : Data Mining, Inference, and Prediction*. Springer, 2 edition, 2009. Section 4.3 (Linear Discriminant Analysis).

[2] W. N. Venables and B. D. Ripley. *Modern Applied Statistics with S*. Springer, New York, 4 edition, 2002. Implements `MASS::lda`.

A Modules exportés

Les fonctions de `stats_utils.R` restent internes. Seule la classe `LDA` est exportée dans le `NAMESPACE`.

B Jeux d'exemple

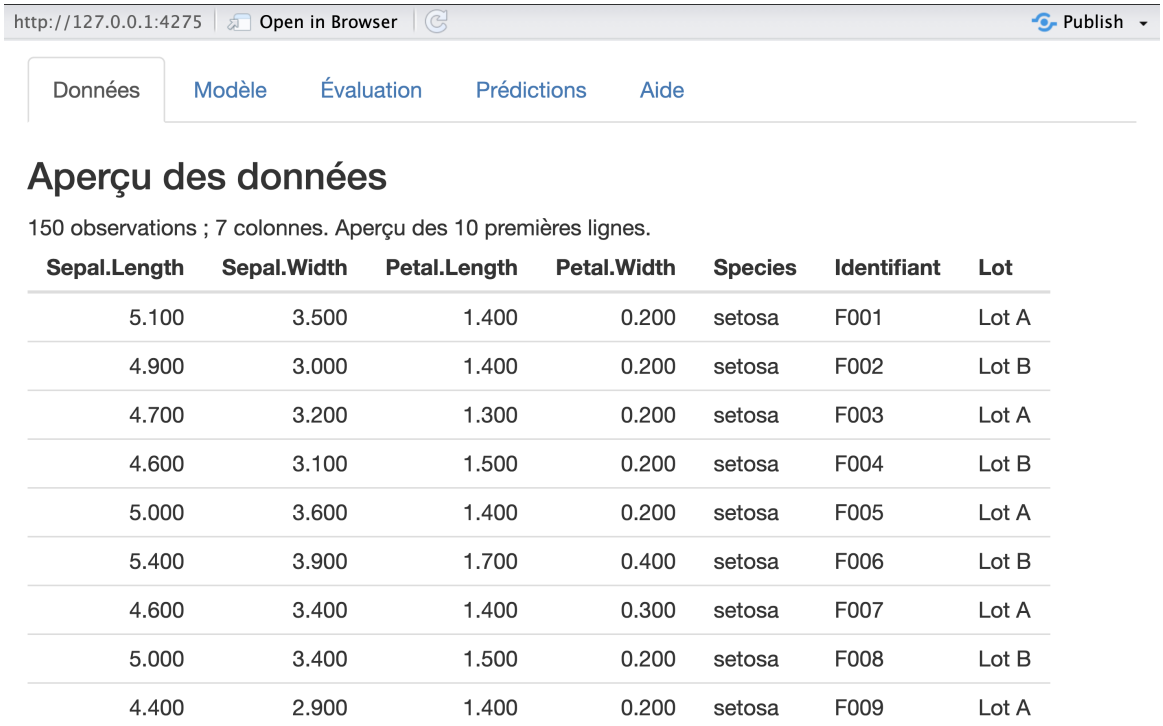
Trois fichiers accompagnent le tutoriel et l'application : `iris_apprentissage.tsv`, `iris_apprentissage_fr.` et `iris_nouveaux.tsv`.

C Captures de l'application Shiny

Les figures suivantes illustrent un parcours sur l'exemple iris fourni dans l'application. Le tutoriel vidéo correspondant est disponible ici : https://youtu.be/QUwlcWtNWKk.



FIGURE 1 – Choix de la source de données (exemple iris ou import de fichier).



Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species	Identifiant	Lot
5.100	3.500	1.400	0.200	setosa	F001	Lot A
4.900	3.000	1.400	0.200	setosa	F002	Lot B
4.700	3.200	1.300	0.200	setosa	F003	Lot A
4.600	3.100	1.500	0.200	setosa	F004	Lot B
5.000	3.600	1.400	0.200	setosa	F005	Lot A
5.400	3.900	1.700	0.400	setosa	F006	Lot B
4.600	3.400	1.400	0.300	setosa	F007	Lot A
5.000	3.400	1.500	0.200	setosa	F008	Lot B
4.400	2.900	1.400	0.200	setosa	F009	Lot A

FIGURE 2 – Aperçu des données chargées.

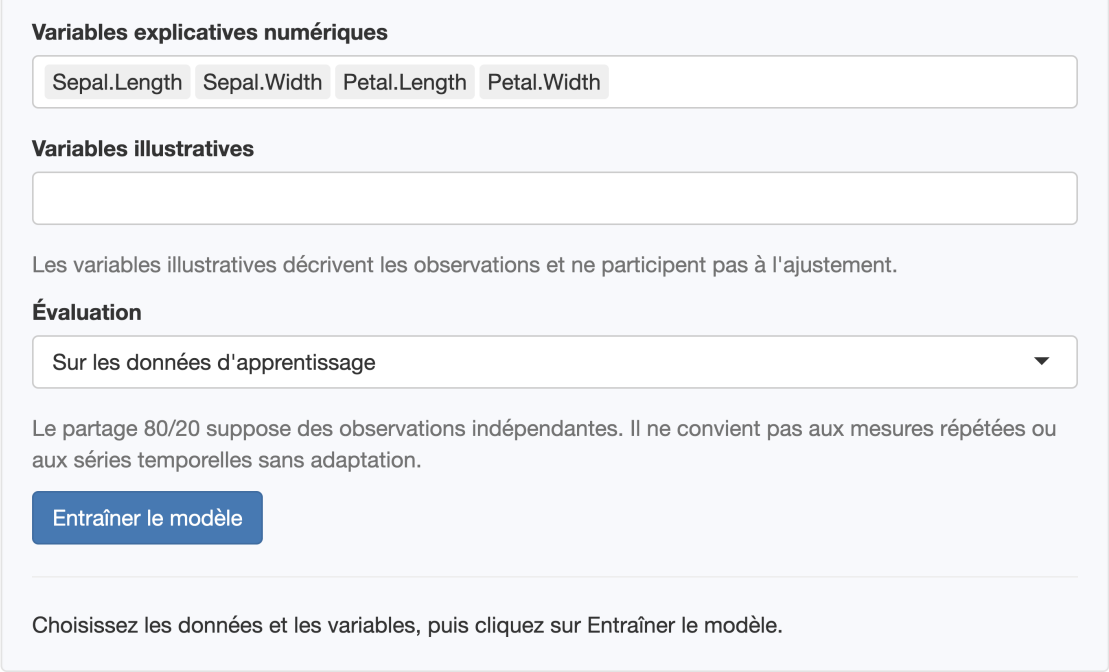


FIGURE 3 – Sélection de la cible, des variables explicatives, des variables illustratives et du mode d'évaluation.

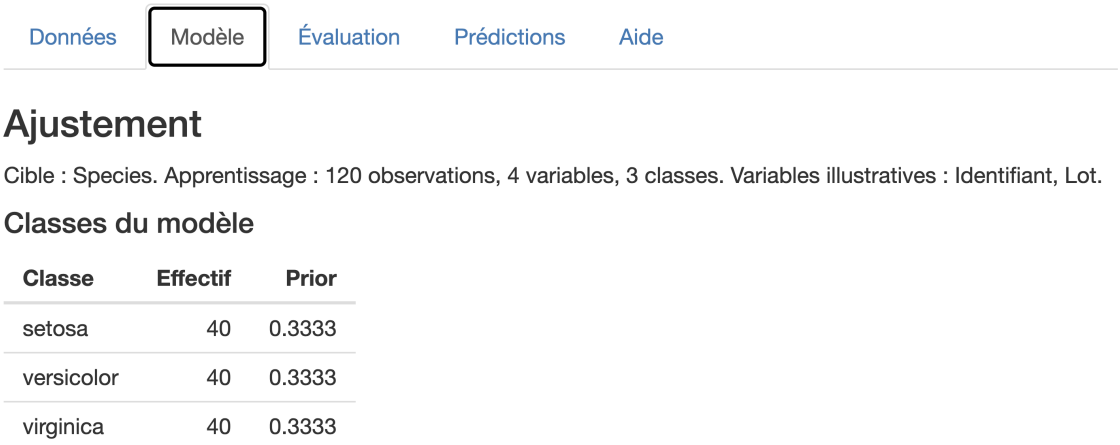


FIGURE 4 – Onglet Modèle : résumé de l'ajustement et répartition des classes.

Coefficients des fonctions de classification

Une colonne par classe, avec l'interception en première ligne. Ces coefficients ne constituent pas un classement d'importance des variables.

Variable	setosa	versicolor	virginica
(Intercept)	-81.6396	-82.7852	-116.2821
Sepal.Length	24.1196	18.8751	16.3400
Sepal.Width	18.9704	3.0186	0.5345
Petal.Length	-14.5072	8.0451	14.4044
Petal.Width	-12.6444	5.2603	20.1922



FIGURE 5 – Coefficients des fonctions de classification.

Moyennes par classe

Classe	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
setosa	5.0050	3.4075	1.4600	0.2450
versicolor	6.0050	2.7600	4.3100	1.3350
virginica	6.5825	3.0075	5.5300	2.0575

FIGURE 6 – Moyennes par classe.

Résultats de classification

Évaluation sur 30 observations de test exclues de l'ajustement ; apprentissage sur 120 observations. Un seul partage stratifié, graine 2026.

Indicateur	Valeur
Exactitude	0.9667
Exactitude équilibrée	0.9667
F1 macro	0.9666

FIGURE 7 – Indicateurs globaux d'évaluation.

Résultats par classe

Classe	Effectif	Precision	Rappel	F1
setosa	10	1.0000	1.0000	1.0000
versicolor	10	0.9091	1.0000	0.9524
virginica	10	1.0000	0.9000	0.9474

FIGURE 8 – Précision, rappel et F1 par classe.

Matrice de confusion : lignes observées, colonnes prédites

Classe_observee	setosa	versicolor	virginica
setosa	10	0	0
versicolor	0	10	0
virginica	0	1	9

FIGURE 9 – Matrice de confusion (lignes observées, colonnes prédites).

Prédire de nouvelles observations

Entraînez d'abord le modèle. La variable cible n'est pas nécessaire pour la prédiction.

Mode de saisie

(•) Un individu ( ) Un fichier

Sepal.Length

5,8

Sepal.Width

3

Petal.Length

4,4

Petal.Width

1,3

FIGURE 10 – Saisie d'une nouvelle observation à prédire.

Calculer les prédictions

Observation	Classe_predite	Probabilite_max	P(setosa)	P(versicolor)	P(virginica)
1	versicolor	0.9997	0.0000	0.9997	0.0003

FIGURE 11 – Résultat de prédiction pour une nouvelle observation.

Utilisation

Choisissez l'exemple iris ou importez un fichier avec une ligne d'en-tête. Vérifiez le séparateur, le symbole décimal et l'aperçu. Pour xlsx, la première feuille est lue ; le package readxl est nécessaire.

Sélectionnez une cible catégorielle comportant au moins deux classes. Les variables explicatives doivent être numériques, sans valeurs manquantes ou infinies. Les variables illustratives peuvent être textuelles ou numériques et restent exclues du modèle.

Cliquez sur Entraîner le modèle. Les changements de données, de variables ou de mode d'évaluation effacent les résultats pour éviter de présenter un ancien modèle.

Dans Prédictions, saisissez un individu ou importez un fichier possédant les mêmes noms de variables explicatives. L'ordre des colonnes peut différer. Les colonnes supplémentaires sont conservées dans le téléchargement.

Évaluation et interprétation

Exactitude : proportion de classifications correctes. Précision d'une classe : proportion d'observations correctes parmi celles prédites dans cette classe. Rappel : proportion d'observations retrouvées parmi celles appartenant réellement à cette classe. F1 : 2TP / (2TP + FP + FN). L'exactitude équilibrée est la moyenne des rappels ; le F1 macro est la moyenne des F1 des classes.

FIGURE 12 – Onglet Aide de l'application.